

Dynamic Task Scheduling and Adaptive GPU Resource Allocation in the Cloud

Hoda Sedighi¹, Fetahi Wuhib², and Roch H. Glitho³, *Life Senior Member, IEEE*

Abstract—The growing demand for computational power in cloud computing has made Graphics Processing Units (GPUs) essential for providing substantial computational capacity. Efficiently allocating GPU resources is crucial due to their high cost. Additionally, it's necessary to consider cloud environment characteristics, such as dynamic workloads, multi-tenancy, and requirements like isolation. One key challenge is efficiently allocating GPU resources while maintaining isolation and adapting to dynamic workload fluctuations. Another challenge is ensuring scheduling maintains fairness between tenants while meeting task requirements (e.g., completion deadlines). While existing approaches have addressed each challenge individually, none have tackled both challenges simultaneously. This is especially important in dynamic environments where applications continuously request and release GPU resources. This paper introduces a new dynamic GPU resource allocation method, incorporating fair and requirement-aware task scheduling. We present a novel algorithm that leverages the multitasking capabilities of GPUs supported by both hardware and software. The algorithm schedules tasks and continuously reassesses resource allocation as new tasks arrive to ensure fairness. Simultaneously, it adjusts allocations to maintain isolation and satisfy task requirements. Experimental results indicate that our proposed algorithm offers several advantages over existing state-of-the-art solutions. It reduces GPU resource usage by 88% and significantly decreases task completion times.

Index Terms—Fair resource sharing, dynamic resource management, task scheduling, GPU resource allocation, efficiency in cloud computing.

I. INTRODUCTION

GPUs, with their extensive parallel computing capabilities, are essential for accelerating compute-intensive tasks in cloud environments. However, the high cost of GPUs necessitates efficient utilization strategies to balance performance and resource efficiency. The diverse nature of cloud task demands introduces challenges in managing GPU allocation, as tasks often require only a fraction of a GPU's full computational

power. Assigning an entire GPU to a single task ensures adherence to Quality of Service (QoS) and Service Level Agreement (SLA) policies, but it frequently results in resource underutilization [1], [2]. Various mechanisms leverage GPU multitasking techniques to address GPU inefficiency, allowing multiple tasks to share the same GPU. While these GPU-sharing mechanisms enhance overall resource utilization and reduce costs, they also introduce new challenges.

The first challenge is that many existing approaches focus on static resource allocation strategies without considering how task requirements change over time. However, in real-world cloud environments, tasks exhibit varying resource demands due to fluctuating computational needs, unpredictable job arrivals, and shifting execution priorities [3]. A static allocation model [4] that does not adapt to these variations can lead to resource underutilization, performance bottlenecks, or severe contention between tasks. Furthermore, while GPU multitasking techniques improve utilization, they introduce additional challenges in maintaining strong isolation between tasks. Software-based multitasking methods [5] enable spatial GPU sharing, offering finer-grained control over resource distribution and improving performance [6]. However, this approach lacks strict isolation, as concurrent tasks may inadvertently access each other's memory or suffer from cascading failures if one crashes. On the other hand, hardware-based GPU partitioning ensures stronger isolation by dividing a GPU into independent units with dedicated resources [7]. Still, its limited granularity makes achieving optimal resource utilization difficult [8]. In a dynamic environment where task requirements continuously change, resource efficiency with isolation constraints necessitates an adaptive allocation strategy. Thus, GPU resource allocation must dynamically adjust to changes in task requirements while optimizing utilization and maintaining isolation.

The second critical challenge in managing GPU resources within a multi-tenant cloud environment is ensuring fair task scheduling. This approach guarantees that each task receives adequate processing time, reduces long wait times, and minimizes the average task completion time [9]. Traditional schedulers, such as First-In-First-Out (FIFO) or MISO [7], struggle to maintain fairness, often resulting in long execution delays for specific tasks. While Shortest-Job-First (SJF) prioritizes shorter tasks to improve overall completion time, it fails to consider tasks' resource demands, potentially leading to inefficient GPU usage. Shorter tasks with high resource requirements leave fewer resources available, preventing other tasks from being executed. The challenge

Received 14 April 2025; revised 28 September 2025; accepted 17 November 2025. Date of publication 19 November 2025; date of current version 29 December 2025. This work is funded by the Ericsson/ENCQOR-5G Senior Industrial Research Chair on Cloud and Edge Computing for 5G and Beyond. The associate editor coordinating the review of this article and approving it for publication was M. Tornatore. (Corresponding author: Hoda Sedighi.)

Hoda Sedighi is with the Concordia Institute of Information System Engineering, Concordia University, Montreal, QC H3G 1M8, Canada (e-mail: h_sedi@encs.concordia.ca).

Fetahi Wuhib is with the Ericsson Research, Ericsson, Montreal, QC H4S 0B6, Canada (e-mail: fetahi.wuhib@ericsson.com).

Roch H. Glitho is with the Concordia Institute of Information System Engineering, Concordia University, Montreal, QC H3G 1M8, Canada, and also with the Computer Science Program, University of the Western Cape, Cape Town 7535, South Africa (e-mail: glitho@ece.concordia.ca).

Digital Object Identifier 10.1109/TNSM.2025.3635529

becomes more complex in dynamic environments where static scheduling policies might no longer be optimal as task requirements change over time [8], [9]. A fair adaptive scheduling approach is required to prevent resource starvation, minimize task waiting times, and ensure balanced GPU utilization.

Given the complexity of GPU resource management in dynamic cloud environments, optimizing both allocation and scheduling remains an NP-hard challenge [10]. Recently, in [11], we proposed a fair mechanism for task scheduling and resource allocation. Reference [11] focused on a static allocation of GPU resources based on predefined task demands, assuming all tasks arrive simultaneously at a designated time. This assumption allows algorithms to schedule all tasks at once. Although [11] effectively addressed resource allocation and ensured task isolation, it did not consider dynamic allocation strategies that adapt to changing task demands. Furthermore, we noted that [11] overlooked the dynamic nature of task demands in cloud environments, where tasks can arrive at different times. Additionally, scheduling and allocating resources without continually reassessing the fairness metric can lead to imbalances in resource distribution. This can result in unfair execution of tasks over time and potentially causing delays in task completion. Another issue with the [11] is that we employed an estimation mechanism to calculate the static delay before scheduling the task. However, achieving fair allocation while continually reassessing fairness requires a dynamic calculation of waiting times. Finally, [11] suffers from the lack of repartitioning, reallocation, and task eviction capabilities, leading to increased resource utilization and longer task completion times. In this work, we build on previous research [11] by introducing a dynamic resource allocation framework that adapts to task demand variations while ensuring both isolation and fairness. Our key contributions are as follows:

- We propose an adaptive allocation strategy that combines hardware-based and software-based partitioning techniques, enabling flexible resource adjustments based on various task demands.
- We develop a scheduling mechanism that dynamically schedules tasks, with repartitioning, reallocation, and task eviction capabilities to optimize resource utilization and minimize task completion times.
- We integrate a monitoring system that collects data on task progress, including execution time and remaining time, as well as GPU partition status and partition release time. This monitoring data is then used to calculate wait time dynamically, continually reassess the fairness metric over time, and efficiently reallocate resources.
- We evaluated the effectiveness of our approach by employing production workload traces from Alibaba's cluster-trace-gpu dataset in a simulated cluster environment, consisting of over 7,000 GPU-dependent task requests distributed across up to 64 GPUs.

By integrating these adaptive strategies, our approach improves GPU efficiency in multi-tenant cloud environments, ensuring fair and effective task execution.

II. RELATED WORK

Efficiently allocating and scheduling GPU resources for multiple users with different task demands is crucial in cloud computing environments. This is especially important when task demands are dynamic. The following sections review existing GPU resource allocation and task scheduling research, emphasizing their advantages and limitations.

A. GPU Resource Allocation

Traditional methods for allocating GPU resources are mainly categorized into temporal and spatial multitasking techniques [5]. Temporal multitasking allows tasks to access GPU resources in specific time slices [5], [12], [13], which ensures isolation between tasks. However, this method often leads to underutilization since only one task can use the GPU at a time. Preemption techniques have been introduced to enhance responsiveness [14], [15], [16]; however, these can result in significant overhead due to the high context-switching costs associated with GPU architectures. Spatial multitasking, conversely, divides GPU resources among concurrent tasks, which boosts utilization and enhances overall system performance [17], [18], [19]. Services such as NVIDIA's Multi-Process Service (MPS) [20] allow different tasks to share GPU resources simultaneously. However, while spatial multitasking improves efficiency, it may compromise isolation between tasks.

Recent advancements have introduced hardware-based sharing techniques, such as NVIDIA's Multi-Instance GPU (MIG) technology, which offers strong isolation by dividing a GPU into separate instances. While MIG provides robust isolation, it supports a limited number of partitions, which can lead to underutilization if the task requirement does not align with the available configurations. For instance, NVIDIA A100 GPUs support up to seven partitions, which may not align with diverse task requirements. To address these challenges, some researchers have proposed methods that utilize hardware-based sharing techniques combined with learning-based performance predictors to estimate task performance on different GPU slices and determine optimal hardware partitions for co-located tasks [7]. However, these approaches only apply to GPUs that support hardware-based sharing technology. Moreover, relying solely on hardware-based technology may lead to GPU underutilization due to a limited number of feasible partitions supported by GPUs.

Recent efforts have focused on improving GPU utilization through adaptive resource control mechanisms. For example, ICEFROG [21] proposes a layer-elastic scheduling system that reduces GPU usage by selectively freezing specific layers of deep learning models during training. This method enables overlapping computation and communication phases, enhancing throughput and increasing the efficiency of GPU resources in cluster environments. However, ICEFROG relies on strong assumptions, such as centralized infrastructure and static task structures, which limit its applicability in cloud-native environments. In contrast, our work targets multi-tenant and dynamic GPU task requirements with minimal assumptions about the task structure or execution model.

Beyond ICEFROG and the static methods mentioned, several notable works address adaptive resource allocation and efficiency. Zico [22] introduces adaptive GPU allocation for concurrent DNN training, reducing memory overhead and improving efficiency compared to naive spatial sharing. However, Zico focuses primarily on alleviating the memory bottleneck in deep learning training, whereas our work addresses end-to-end fairness and efficient allocation across diverse tenants using hardware partitioning and spatial multitasking. MuxFlow [23], deployed at scale on 20,000+ GPUs, proposes a two-level protection mechanism (xCUDA + SysMonitor) and matching-based scheduling to guarantee safe space-sharing of online/offline workloads in production clusters. Unlike MuxFlow, which is tailored to protecting latency-critical inference jobs in large-scale production environments, our approach targets heterogeneous multi-tenant cloud workloads and emphasizes fairness with isolation guarantees. EasyScale [24] enables accuracy-consistent adaptive resource allocation in training by decoupling DL training behaviour from GPU allocation, boosting GPU utilization. In contrast, EasyScale is limited to deep learning training jobs, whereas our solution is designed for general cloud-native tasks, requiring no workload-specific modifications. Finally, ntaskset [25] introduces a hardware-backed fine-grained spatial partitioning tool for real-time and composable systems, achieving low overheads and strong isolation without task modifications. While highly effective for safety-critical embedded workloads, it is less applicable to adaptable, multi-tenant cloud environments that our approach targets.

Despite these advances, existing approaches still struggle to balance fairness and efficiency in heterogeneous GPU clusters. OEF [26] introduces an allocation framework that integrates fairness guarantees with efficiency to maximize training throughput and shorter completion times. While highly effective in DL-centric clusters, OEF's design remains primarily tailored to training workloads and depends on accurate job profiling to inform allocation decisions. In a different application domain, [27] proposes a fair and efficient allocation scheme for cloud rendering workloads. Their method introduces weighted acceleration ratio fairness, which ensures that the speedup of each job is proportional to its SLA-defined weight. By combining fairness metrics with efficiency-oriented resource allocation, their algorithms outperform traditional max-min and proportional fairness baselines. However, this approach is specialized for parallel rendering workloads and does not directly address heterogeneous multi-tenant GPU allocation problems. In contrast, our approach targets diverse, dynamic workloads in cloud-native multi-tenant environments. Unlike OEF [26] and rendering-focused models [27], it adapts resources in real time without offline profiling. By integrating hardware partitioning with software multitasking, our approach guarantees isolation, prevents underutilization, and achieves both fairness and efficiency.

B. Dynamic GPU Task Scheduling

In dynamic cloud environments where task demands fluctuate, schedulers must adapt to changing conditions. Several

studies have focused on runtime scheduling techniques such as auto-scaling, sorting, task preemption, and workload-aware scheduling to optimize GPU resource utilization and meet resource requirements. The proposed scheduling technique [28] sorts real-time tasks by their arrival time to maximize success rates while meeting deadlines. However, it can result in longer completion times and unfair scheduling due to delays in evaluating all options and neglecting task waiting times. MISO [7] employs FIFO scheduling to enhance overall system performance by minimizing task disruption through GPU hardware-based sharing. However, relying solely on this hardware limits partitioning, resulting in fewer concurrently executed tasks and increased completion times. The TGS scheduling technique in [29] maximizes task throughput by controlling the task dequeuing rate. However, this method results in longer waiting and completion times, leading to unfair scheduling.

Recent research on GPU task scheduling has shifted towards fair scheduling approaches, notably the min-max fairness policy, which maximizes the minimum resources allocated to each task [30]. Tasks are weighted based on factors such as deadlines or priorities. However, many current fairness scheduling methods rely on temporal and spatial multitasking techniques, which can lead to time-consuming preemption and inadequate task isolation [9], [31]. Tiresias [31] ensures fairness by minimizing average task completion time and using preemption to prevent short tasks from being delayed by larger ones. However, this approach incurs overhead from frequent data transfers to and from GPU memory and can lead to increased completion times due to continuous preemptions. Moreover, while Tiresias is effective for workloads dominated by short jobs, its preemption-based design can lead to significant inefficiency for long-running or memory-intensive tasks, as they suffer from context-switching and data migration. In contrast, our method provides fairness without relying on preemption. Instead, it incorporates both task completion time and resource utilization into the fairness metric. This approach reduces scheduling overhead for short tasks while still preventing starvation of longer ones, resulting in a more balanced tradeoff between efficiency and fairness.

Themis [32] employs a bidding system for fairness scheduling to minimize maximum task completion time. Still, it is ineffective for short-running tasks, as they create significant allocation and de-allocation overhead. Moreover, Themis focuses solely on completion time, neglecting resource requirements. Its bidding mechanism ensures fairness among multiple tenants, but adds negotiation complexity and does not consider GPU fragmentation. This can lead to suboptimal resource utilization in environments with diverse workloads. In contrast, our approach integrates resource demand awareness directly into the fairness metric, allowing efficient allocation across diverse workloads without the overhead associated with bidding. While Themis can sometimes reduce completion time more aggressively, our method offers more stable throughput and fairness in multi-tenant cloud environments, where minimizing fragmentation and overhead is crucial. FGD [33] proposes a scheduling method that minimizes fragmentation and optimizes allocation rates by packing tasks. However, its

reliance on software techniques restricts resource utilization and prevents task packing among different users. As a result, it fails to achieve maximum resource usage due to isolation policies in a cloud environment.

In the domain of dynamic task scheduling, notable progress has been made toward interference-aware execution models. PBS [34] introduces a scheduling framework for real-time GPU tasks that utilizes a scheduling graph to identify feasible paths for parallel and sequential execution. By analyzing task interference and employing offline schedulability analysis, PBS ensures that tasks meet their deadlines while maximizing parallelism. However, the PBS approach assumes complete knowledge of task arrival patterns and the absence of cross-tenant interference, conditions that are not realistic in cloud-based GPU environments. These limitations restrict its adaptability to dynamic task demands in the cloud environment.

Complementary to PBS, which assumes complete knowledge of task arrivals, FFT [35] proposes a scheduling system that reduces job completion times (JCT) while preserving fairness in heterogeneous GPU clusters. FFT leverages a fine-grained allocation vector and integrates a fairness compensation mechanism, allowing it to adapt to migration overheads and dynamic cluster conditions. Nevertheless, FFT remains optimized for distributed DL training jobs, requires profiling to capture workload characteristics, and does not explicitly target general multi-tenant cloud workloads. More broadly, [36] explores game-theoretic approaches to resource scheduling, demonstrating alternative formulations of the fairness–efficiency tradeoff. The study compares Nash bargaining, Shapley value, Nash equilibrium, congestion games, and Vickrey–Clarke–Groves (VCG) auctions. Although insightful, these methods remain abstract and are primarily evaluated on artificial workloads, without mechanisms for real-time adaptation to dynamic workloads. In contrast, our approach is specifically designed for dynamic workloads that eliminate the need for offline profiling and adapt to runtime workload changes.

More recent systems co-optimize scheduling with workload adaptability. Pollux [37] introduces goodput-driven co-adaptive scheduling, dynamically tuning batch size and learning rate along with GPU allocations. This reduces average task completion times and improves fairness. DCUDA [38] proposes multi-resource-aware scheduling with live migration, reducing execution times with low migration overhead. MuxFlow [23] further integrates matching-based scheduling to balance online and offline workloads in production-scale clusters. These works advance adaptive, fairness, and utilization, but most assume DL-centric workloads [24], [37], production inference clusters [23], or migration-friendly CUDA applications [38]. In contrast, our approach targets heterogeneous, multi-tenant cloud-native workloads, emphasizing fairness and adaptability without requiring workload modifications or specialized infrastructure.

Researchers in edge and approximate computing have explored GPU resource allocation to enhance energy efficiency through adaptive scheduling. sBEET [39] balances real-time performance and energy consumption by dynamically tuning

multitasking configurations, while Zhai et al. [39] propose an equipment–edge–cloud framework that jointly minimizes task completion time and energy use. Approximate computing offers another angle, reducing execution time and GPU demand by tolerating bounded inaccuracies. Singh and Nasre [40] apply heuristic approximations to accelerate graph processing, and Fink et al. [41] extend this idea to HPC scheduling with HPAC-Offload, improving efficiency across GPU clusters. Leon et al. [42] further survey cross-layer approximation methods, showing how they can be integrated into allocation frameworks to balance throughput and energy. While these approaches improve utilization, throughput and reduce costs, they depend on approximation tolerance or edge-specific contexts. In contrast, our work targets general-purpose, multi-tenant cloud environments, where fairness, isolation, and efficiency must be achieved without approximation.

III. SYSTEM MODEL AND PROBLEM STATEMENT

Modern cloud platforms offer GPU-accelerated computing environments that allow multiple users to execute diverse tasks. Cloud users, especially those engaged in high-demand tasks like data processing, machine learning, large language models (LLMs), and generative AI tasks, rely on GPU resources for both training and inference. Figure 1a illustrates a simplified cloud system where users access resources in a data center with several nodes. Many nodes are equipped with GPUs to enhance processing capabilities. Each node usually has multiple GPUs, allowing for efficient high-performance computing execution. Each user has a unique set of tasks that are color-coded for differentiation, and these tasks arrive in the system at different times. For example, green represents tasks for user 1, blue indicates tasks for user 2, and so on. Each task has distinct resource requirements. This variety of tasks and their specific needs highlights the complexity of the cloud environment. Users can specify their GPU requirements alongside CPU characteristics (e.g., memory, type and number of CPU cores).

As shown in Figure 1a, each task is linked to specific GPU resource requirements, with a maximum resource demand defined by cloud users [43]. Users cannot exceed the maximum resources requested, and cloud resource providers must ensure they meet these limits. Moreover, tasks have varying completion times, and instead of relying on time-consuming online profiling [44], [45], modern methods utilize proactive prediction based on task features [46]. This allows for immediate estimation of execution time upon task submission, facilitating prompt task scheduling without delays. As shown in Figure 1a, task characteristics vary in GPU resource needs and execution time. Long-term training of large AI models fully utilizes GPUs by processing hundreds of samples in parallel [32]. In contrast, short-term tasks, like medical imaging enhancement, may only need a fraction of GPU resources, resulting in underutilization [9]. A task scheduler is essential for assigning appropriate GPUs and optimizing resource allocation to enhance utilization while ensuring fairness.

In cloud environments, orchestration tools like Kubernetes enable GPU access and scheduling through device plugins [47]. These plugins provide information on available GPUs

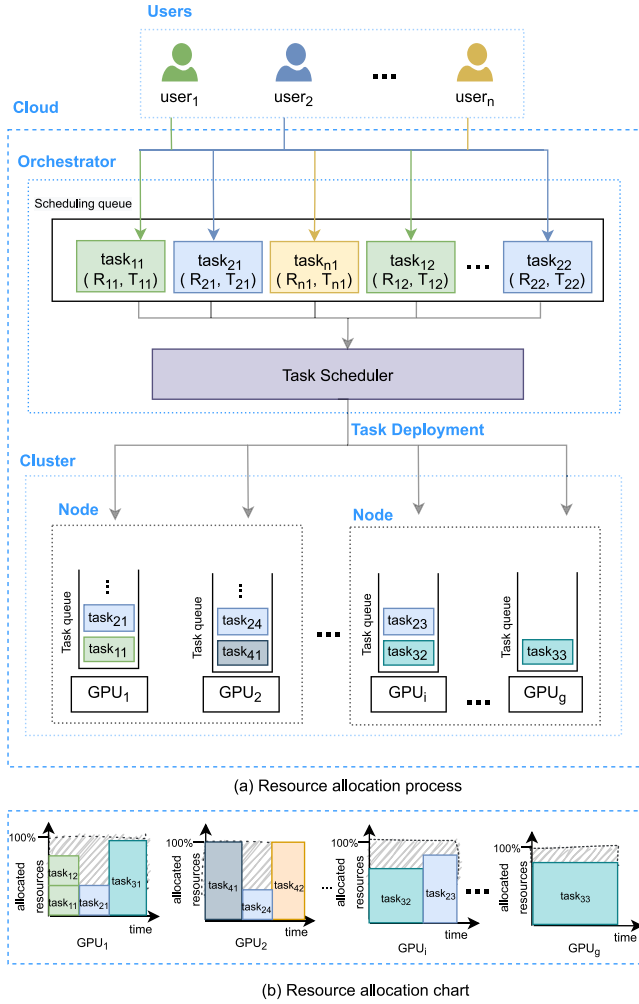


Fig. 1. System model.

and support various sharing technologies, including software and hardware-based technologies. The orchestrator selects a node based on resource availability and allocates a GPU for tasks using scheduling policies like first-fit or Round-Robin for even distribution. The resource allocation charts in Figure 1b illustrate the usage of shared GPU resources in the node. Each chart shows the resource requirements for tasks, with the gray area representing unused GPU capacity over time. This highlights the need for a mechanism to minimize wasted resources, as the default scheduling policy can increase unused capacity. First-fit or even resource assignments lead to greater gray areas, resulting in longer application completion times (x-axis) and increased resource underutilization (y-axis).

A. System Model

We model our cloud environment as a cluster containing a set of G GPUs. Each specific GPU, denoted as g , can be partitioned into a maximum of K_g hardware partitions. These partitions can be allocated to process user tasks, enabling efficient GPU services. The cloud environment serves N users, each requiring access to the shared GPU resources. Every user has a specific set of tasks to execute on the

cloud system. Let M_n represent the total number of tasks for user n . For each task m belonging to user n , R_{mn} indicates the resource requirements. Additionally, each task has a dynamic arrival time, A_{mn} , which refers to the moment user n submits task m . Finally, T_{mn} indicates the estimated completion time for task m belonging to user n , reflecting the anticipated duration needed to finish the task once it starts running with the allocated resources. In practice, T_{mn} is not provided by the user but is estimated by the cloud management system. Rather than relying on time-consuming online profiling [44], [45], modern methods utilize proactive prediction techniques based on task features (e.g., resource requests, input size, or historical runtimes of similar jobs) [46]. This enables immediate estimation of execution time upon submission, allowing the scheduler to make prompt allocation decisions. While the actual completion time may deviate due to contention or preemption, maintaining T_{mn} as a predictive baseline supports efficient and adaptive resource allocation.

In this work, we define a task as the smallest schedulable unit that executes on a single GPU or GPU partition. Multi-GPU jobs, such as large-scale LLM training, can therefore be modelled as a collection of independent tasks, each scheduled on a separate GPU resource. This assumption allows us to focus on fine-grained scheduling decisions while still representing workloads that require multiple GPUs through multiple concurrent tasks.

B. Problem Statement

This section formalizes the resource allocation model for a multi-tenant cloud environment that prioritizes isolation. In this setting, various users aim to enhance their task performance by utilizing shared GPU resources in a data center. Our goal is to dynamically allocate these shared GPU resources through a combination of hardware and software partitioning techniques, optimizing GPU utilization while ensuring that isolation requirements are satisfied. We also take into account the dynamic nature of task demands, acknowledging that tasks can arrive at varying times with diverse resource requirements. This necessitates real-time adjustments in resource allocation to accommodate incoming tasks, release resources from completed ones, preempt tasks to free resources as needed and optimize overall GPU efficiency. Our goals also include fair task scheduling by minimizing the completion time while continuously adapting to changes in task demands. In the following, we outline our problem formulation. A summary of these notations can be found in Table I.

Input Parameters: Building on the definitions in Section III-A, we now define the additional parameters relevant to the problem formulation:

- S_{mn} : The time when the execution of task $m \in M_n$ for user $n \in N$ starts after the necessary resources have been provided to it.
- F_{mn} : The time when the task $m \in M_n$ for user $n \in N$ is completed, after which it frees up the resources.
- T_{mn} : The estimated time of completion task $m \in M_n$ for user $n \in N$ reflects the anticipated time required to

TABLE I
NOTATIONS USED IN THE PROBLEM STATEMENT

Notation	Description
G	The total number of GPUs that are available in the cluster
N	The total number of users utilizing shared cluster resources
M_n	The number of tasks submitted by each user n
R_{mn}	The resource needs for task $m \in M_n$ belonging to user $n \in N$
A_{mn}	The arrival time of a task when the user $n \in N$ submits task m
S_{mn}	The time when the execution of task $m \in M_n$ for user n starts
F_{mn}	The time when the task $m \in M_n$ for user $n \in N$ is completed
T_{mn}	The estimated time of completion task $m \in M_n$ for user $n \in N$
$W_{mn}(t)$	The waiting time for task m of user n at a given time t
C_{mn}	The time taken to complete task $m \in M_n$ for user $n \in N$
K_g	The number of hardware partitions in a specific GPU $g \in G$
$y_{n,m,k,g}(t)$	Binary decision variable for allocating partition k to task m

finish task m once it starts running with the resources that have been allocated. This time spans from S_{mn} to F_{mn} .

- $W_{mn}(t)$: A proactive prediction determines T_{mn} as the anticipated execution time of a task when it operates in an isolated environment with a dedicated GPU. Nevertheless, the actual completion time of the task can vary significantly when tasks share GPU resources. This variation is caused by delays associated with assigning shared resources and the placement of tasks in the scheduling process [29]. We define $W_{mn}(t)$ as the metric that indicates the delay in scheduling placement. This metric reflects the waiting time for task $m \in M_n$ of user $n \in N$ at a given time t . The total duration of W_{mn} includes the interval from when the task is submitted A_{mn} until it begins execution S_{mn} , while the task remains pending. It also encompasses the period during which the task is halted due to resource eviction until the resources can be reassigned and execution can resume.
- C_{mn} : The time taken to complete task $m \in M_n$ for user $n \in N$ is defined as the duration between A_{mn} and F_{mn} . This duration comprises the waiting time before the task is scheduled and the anticipated execution time. This metric reflects how an unfair scheduling policy can result in increased waiting times and, as a result, longer completion times, which is expressed by the formula $C_{mn} = W_{mn} + T_{mn}$.

Decision Variables:

- $y_{n,m,k,g}(t)$: Is a binary decision variable that takes the value of 1 if task $m \in M_n$ of user $n \in N$ is allocated a hardware partition k of GPU $g \in G$ at time t ; it is zero otherwise.

Objective Function: Our goal is to minimize the maximum normalized completion time across all tasks while considering their resource demands and estimated completion times. By focusing on reducing the maximum normalized completion time, we aim to prevent significant delays for any single task compared to others. This approach is crucial for ensuring fair resource allocation and avoiding situations in which some tasks finish far earlier than others. The normalized completion time is defined as the total completion time of a task, including both waiting and execution time, divided by its estimated execution time. A normalized completion time close to 1 indicates that the task was completed as expected, while higher values suggest delays caused by resource contention or

scheduling inefficiencies. The normalized completion time for a given task N_{nm} is calculated as follows:

$$N_{nm} = (C_{mn}/T_{mn}), \quad (1)$$

We define the continuous version of the normalized completion time to evaluate task execution efficiency over time. Given that there are $\sum_{n=1}^N M_n$ active tasks at a specific time t within the cluster, we define the continuous version of the normalized completion time for any task $m \in M_n$ belonging to users $n \in N$ as follows:

$$N_{nm}(t) = (C_{mn}(t)/T_{mn}) \sum_{g=1}^G \sum_{k=1}^{K_g} y_{n,m,k,g}(t) \cdot I(t \geq A_{mn}), \quad (2)$$

This definition is used to determine the maximum normalized completion time, which serves as an objective for optimization. The indicator functions $I(t \geq A_{mn})$ ensures that only tasks that have arrived by time t are considered for scheduling. The algorithm executes iteratively upon the arrival of new tasks. If the cluster operates for a time interval of Δ , we can determine the maximum normalized completion time, serving as our objective function, using the following formula:

$$\text{Minimize : } N_{\Delta} = \int_0^{\Delta} \max_{1 \leq n \leq N} \max_{1 \leq m \leq M_n} N_{nm}(t) dt, \quad (3)$$

Constraints: We implement constraint (4) as a cloud isolation measure to guarantee that each task is allocated exclusively to a single hardware partition during the time interval Δ :

$$\int_{A_{mn}}^{\Delta} \sum_{g=1}^G \sum_{k=1}^{K_g} y_{n,m,k,g}(t) dt \leq 1, \quad \forall n \in N, m \in M_n \quad (4)$$

The integration begins with the arrival time A_{mn} to reflect the dynamic nature of task arrivals. Additionally, we define constraint (5) to ensure that the resource requirements of all tasks $m \in M_n$ for each user $n \in N$ are adequately met.

$$\sum_{n=1}^N \sum_{m=1}^{M_n} R_{nm} \leq \int_0^{\Delta} \sum_{g=1}^G \sum_{k=1}^{K_g} R_{nm} \cdot y_{n,m,k,g}(t) dt, \quad (5)$$

In addition we presents (6) to model the waiting time accumulates for unscheduled tasks:

$$W_{nm}(t + \Delta t) = W_{nm}(t) + \Delta t \cdot \left(1 - \sum_{g=1}^G \sum_{k=1}^{K_g} y_{n,m,k,g}(t) \right), \quad (6)$$

Finally, this constraint ensures tasks are not scheduled before their arrival time.

$$y_{n,m,k,g}(t) = 0, \quad t < A_{mn} \quad (7)$$

IV. PROPOSED ALGORITHM

Our proposed solution encompasses three primary sub-algorithms: (i) Task grouping, (ii) GPU partitioning, and (iii) Task placement. An overview of this approach is illustrated in Figure 2.

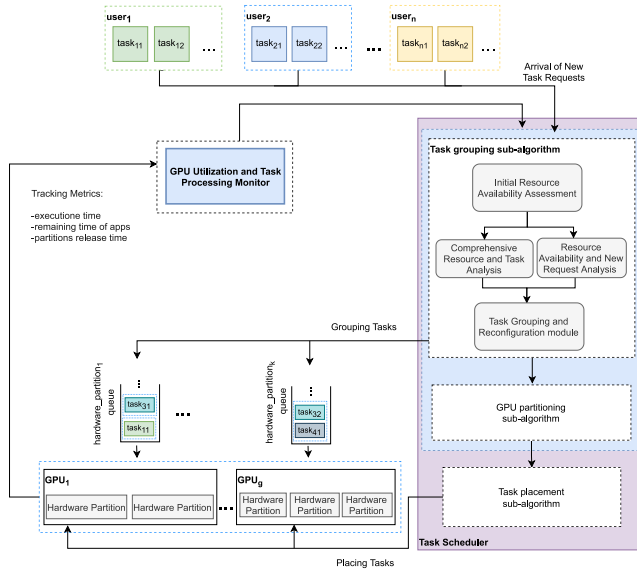


Fig. 2. Overview of the proposed method.

A. Task Grouping

The Task Grouping Sub-Algorithm addresses the challenges of user isolation by calculating the necessary number and size of hardware partitions for each user using hardware partitioning. It also tackles inefficiencies in resource allocation by grouping and co-locating trusted tasks within a single partition through software-based techniques. This sub-algorithm receives the resource needs and completion time of each user's tasks as inputs to identify the most suitable hardware partition and effectively group compatible tasks for co-location. The process of selecting a particular combination of tasks, which vary in resource requirements and execution times, is similar to bin packing, a well-known NP-hard problem. Additionally, GPUs have constraints on partition sizes, and our proposed solution aims to optimize GPU resource consumption by minimizing the total number of partitions utilized. Alongside reducing resource consumption, we strongly emphasize minimizing the normalized task completion time and promoting fair resource distribution. To solve this optimization problem, we employ the Linear Programming (LP) approach.

Algorithm 1 depicts the Task grouping sub-algorithm. The algorithm aims to find the optimal hardware partition and group tasks to optimize GPU resource consumption. Upon receiving a new task or a set of new tasks, along with the relevant resource and time information (r, t), these tasks are placed in a queue for subsequent processing. Initially, the system gathers data on currently running tasks, which includes the release times of the GPU partitions in use, available resources, execution times, and the remaining time for running tasks (lines 3-8). This data is essential for determining how to accommodate the new tasks efficiently. A monitoring module is integrated into the system to gather task progress data and GPU partition status. The system then employs two routines to assess the normalized completion time as the fairness metric for optimal task placement and resource allocation. The two routines can run concurrently, leading to faster execution.

Algorithm 1 Task Grouping With Two-Routine Evaluation

- 1: **Inputs:** N List of users, M_n Number of tasks per user, R_{mn} Resource requirement for each task, T_{mn} Estimated execution time for each task, K Number of possible partitions in GPU, C_j Capacity of each hardware partition j , E_{mn} Remaining execution time of each task, W_{mn} Waiting time for each task, A_{mn} arrival time of each task
- 2: **Output:** y_{mj} List of selected hardware partitions for each task and x_j the number of times hardware partitions j is selected
- 3: **Step 1: Collect System State Information**
- 4: **for** ($n = 1$ to N) **do**
- 5: **for** ($m = 1$ to M_n) **do**
- 6: Retrieve remaining execution time E_{mn} , waiting time W_{mn} , and assigned partitions.
- 7: **end for**
- 8: **end for**
- 9: **Step 2: Execute Two Routines**
- 10: **//Routine 1:** Call Direct Allocation (No Partition Modifications) with new set of submitted tasks and idle resources.
- 11: $\text{min-norm-completion-time-1} = \text{Task-grouping-sub-algorithm Algorithm 2}$;
- 12: **//Routine 2:** Call Reallocation and Repartitioning with all submitted tasks and all resources
- 13: $\text{min-norm-completion-time-2} = \text{Task-grouping-sub-algorithm Algorithm 2}$;
- 14: **Step 3: Compare and Apply Best Solution**
- 15: Select the routine with the best fairness metric.

In Routine 1, if there are enough idle resources, the NP problem is addressed by calculating the normalized completion time based on the new tasks and considering only the available idle resources (lines 10-11). Routine 2, on the other hand, takes into account all GPU resources, whether they are free or in use, leading to a reassessment of the normalized completion time for both existing and newly introduced tasks (lines 12-13). Both strategies consider different scenarios and employ the task grouping sub-algorithm 2 to solve the NP problem. In Routine 1, which focuses solely on idle resources, new tasks can be assigned without any adjustments. However, in Routine 2, where all resources are considered, including those that are currently occupied, it may be necessary to repartition or evict some tasks. After comparing the outcome of evaluating the fairness metrics for both routines, the algorithm will select the optimal solution for grouping tasks (lines 14-15). In this context, the fairness metric refers to the normalized completion time defined in Equation (9), which balances task execution time and waiting time. Both routines are assessed using this metric, and the one that results in the lower normalized completion time will be selected as the final allocation strategy. The scheduling framework is invoked in an event-driven manner. Specifically, Algorithm 1 is triggered upon (i) the arrival of new tasks and (ii) the completion of running tasks that free GPU resources. This ensures responsiveness without requiring periodic global re-optimization.

The Task grouping sub-algorithm, illustrated in Algorithm 2, is designed to group tasks for each routine and find the optimal hardware partition for the grouped task. It classifies incoming tasks according to the user requesting them. Subsequently, the algorithm operates concurrently for all users to guarantee efficient and prompt handling of each request (lines 3-22). Throughout this process, the algorithm takes into account the hardware partitioning constraints regarding both the quantity and size of the partitions. Specifying the restricted partition size options that the particular GPU model supports is crucial. Consequently, we define K as the number of potential partitions available in the GPU, which can differ depending on the model. We represent C_k as the resource of a specific

hardware partition k , where $k \in K$. The algorithm executes a nested iteration to allocate an appropriate partition for each task. For each task (line 4), the process examines all potential hardware partition options (line 5). We define the formula of selecting the most suitable partition for each task with the goal of minimizing resource wastage in each partition as follows:

$$\text{Minimize} : \sum_{k=1}^K \left(C_k - \sum_{m=1}^{M_n} R_{nm} \cdot y_{mj} \right) \cdot x_k \cdot \alpha_k \quad (8)$$

In Equation (8), the variable x_k represents how often a certain hardware partition $k \in K$ is chosen. Additionally, a binary decision variable y_{mk} is created to identify the specific options that reduce resource waste. The algorithm assesses each option with this formula to determine if it effectively minimizes Equation (8) (line 12).

By minimizing resource waste as outlined in Equation (8), this method allows tasks without isolation requirements to be co-located within the same hardware partition. This strategy optimizes the use of available space in a partition, avoids the need for separate hardware partitions for each task, and prevents resource underutilization. Moreover, Equation (9) (line 13) represents a normalized measure of time utilization for a task, considering both execution and waiting times from the task's arrival time A_{mn} until the current time t . This value serves as an indicator of fairness in scheduling. Minimizing this value guarantees that tasks receive appropriate execution opportunities while considering their waiting periods, preventing unfair delays in multitasking environments. For each potential hardware partition option, the corresponding variable y_{mk} is set to 1 if task $m \in M_n$ for user $n \in N$ is allocated to partition j , which occurs when it minimizes both Equation (8) and Equation (9); otherwise, it is assigned 0 (lines 14-19).

$$\text{Minimize} : \left(\frac{(T_{mn} - E_{mn}(t)) + W_{mn}(t)}{T_{mn} - E_{mn}(t)} \right) \cdot y_{mk} \cdot \alpha_k \quad (9)$$

When tasks are grouped together, hardware partition underutilization may occur if the completion times of the tasks within a partition vary. In this situation, even though some resources within the partition can be released earlier, other tasks from different users must wait until all the tasks in that partition are completed before they can access those resources. As a result, the algorithm should group each user's tasks with similar execution times to minimize resource waste for a given hardware partition size. To effectively organize tasks with close execution times within a single partition, we utilize the Time Range Ratio (TRR) metric. This metric calculates the ratio range of task execution times within each partition (line 6). The TRR_k value, which is defined for $k \in K$, lies within the range of 0 to 1 and is calculated using the following formula:

$$TRR_k : \frac{\left(\max_{m=1}^{M_n} T_{nm} \cdot y_{mk} \right) - \left(\min_{m=1}^{M_n} T_{nm} \cdot y_{mk} \right)}{\left(\max_{m=1}^{M_n} T_{nm} \cdot y_{mk} \right)} \quad (10)$$

TRR_k is designed to generate a binary variable α_k , which reflects the influence of task execution time (lines 7-11). The algorithm assesses the execution time of each task against

a defined threshold (Thr). This process aids in ruling out partition choices that contain tasks with a significantly high ratio range.

$$\alpha_k = \begin{cases} 1 & \text{if } TRR_k < Thr \\ 0 & \text{if } TRR_k > Thr \end{cases} \quad (11)$$

The threshold mechanism is intended to balance optimal resource usage and fair distribution. By raising the threshold value (Thr) towards 1, resource utilization is enhanced by allowing more partitioning options. Conversely, decreasing the threshold value to zero limits the number of partition choices but aids in removing low ratio rate options, leading to better task completion times. Once this phase concludes, tasks from each user are grouped to fit within designated hardware partition sizes, forming k hardware partition queues for the grouped tasks.

It's important to understand that the task grouping algorithm is designed under the assumption that tasks from the same user can be safely grouped together within a single hardware partition. This reflects typical multi-tenant cloud environments, where isolation is only enforced between different users. However, if there is a need for isolation within a single user's tasks (for security or confidentiality reasons, for example), this can be addressed by adding an extra constraint to the optimization formulation. In such cases, tasks that require isolation can be assigned to separate partitions by adjusting the binary assignment variables y_{mk} . This ensures that these tasks are not co-located. Although introducing these constraints may lead to decreased partitioning efficiency, the overall framework remains effective and continues to optimize both fairness and resource utilization within the available solution space.

Due to restrictions on GPU partition sizes, resource capacities, and fairness metrics, the task-grouping problem is classified as an NP-hard problem. To achieve feasible and optimal solutions, Algorithm 2 is developed as a model to be solved using an appropriate solver. In the worst-case scenario, solving this model requires exploring an exponential number of combinations of the binary assignment variables (y_{mk}), leading to the time complexity:

$$O(2^{M \cdot K}) \quad (12)$$

where M is the number of tasks, and K is the number of admissible GPU partitions. Algorithm 1 invokes Algorithm 2 twice: once with the newly submitted task set ($M_{\text{new}} \times K$) and once with the complete set of tasks ($M_{\text{all}} \times K$). Its overall complexity is dominated by the second call, resulting in:

$$O(2^{M_{\text{all}} \cdot K}) \quad (13)$$

It is important to note that in practical deployments, the calculation is performed over the currently existing applications, which are typically much smaller than M_{all} . This significantly reduces the effective problem size handled by the solver at runtime. Furthermore, modern solvers such as Google OrTools Solver are able to significantly reduce the search space compared to the theoretical exponential bound by employing branch-and-bound, cutting planes, and conflict-driven learning.

Algorithm 2 Task Grouping Sub-Algorithm

```

1: Inputs:  $N$  List of users,  $M_n$  Number of tasks per user,  $R_{mn}$  Resource requirement
   for each task,  $T_{mn}$  Estimated execution time for each task,  $K$  Number of possible
   partitions in GPU,  $C_k$  Capacity of each hardware partition  $k$ ,  $E_{mn}$  Remaining
   execution time of each task,  $W_{mn}$  Waiting time for each task,  $A_{mn}$  arrival time
   of each task
2: Output:  $y_{mk}$  list of selected hardware partitions for each task and  $x_k$  the number
   of times hardware partitions are selected
3: for ( $n = 1$  to  $N$ ) do
4:   for ( $m = 1$  to  $M_n$ ) do
5:     for ( $k = 1$  to  $K$ ) do
6:        $TRR_k \leftarrow$  calculate based on Equation.10
7:       if ( $TRR_k \leq Thr$ ) then
8:          $\alpha_k \leftarrow 1$ ;
9:       else
10:         $\alpha_k \leftarrow 0$ ;
11:       end if
12:        $Minimize: \sum (C_k - \sum R_{nm} \cdot y_{mj}) \cdot x_k \cdot \alpha_k$ ;
13:        $Minimize: \left( \frac{(T_{mn} - E_{mn}(t)) + W_{mn}(t)}{T_{mn} - E_{mn}(t)} \right) \cdot y_{mj} \cdot \alpha_k$ ;
14:       if option  $k$  fits to minimize the values then
15:          $y_{mj} \leftarrow 1$ ;
16:          $x_k \leftarrow x_k + 1$ ;
17:       else
18:          $y_{mj} \leftarrow 0$ ;
19:       end if
20:     end for
21:   end for
22: end for

```

As a result, the empirical solver runtime often scales closer to polynomial time. In practice, Algorithm 1 can be executed in:

$$O(A + M_{all}K) \quad (14)$$

where A is the number of active tasks. In addition, Algorithm 2 can be observed to run in approximately:

$$O(MK), \quad (15)$$

Consequently, although both algorithms exhibit exponential worst-case complexity when solved exactly via a solver, they also admit polynomial-time heuristic execution that achieves efficient performance in practice.

B. GPU Partitioning

The GPU partitioning sub-algorithm is designed to identify the most efficient way to partition available GPUs using selected hardware partitions, minimizing resource wastage. It takes into account restrictions related to partition fragmentation imposed by GPU policies, including limits on the number of and allowable hardware partition combinations. Algorithm 3 details this routine, emphasizing its goal of optimizing GPU resource utilization.

Determining the best configurations for hardware partitions is difficult because of the complex constraints involved in selecting valid combinations. This issue can be viewed as a variant of an NP-hard problem. The GPU partitioning algorithm employs an LP solver to find the optimal selection of hardware partitions. This algorithm takes as inputs the list of chosen hardware partitions denoted as Y , the frequency of each selected partition $k \in Y$ from the task grouping sub-algorithm, and the list of potential configurations for hardware partitions, CN (line 1). Each configuration of hardware partitions represented as $c \in CN$, is made up of multiple partitions denoted by D_c . The quantity and size of each partition i within c are specified by N_{ic} and R_{ic} . To identify the optimal solution,

Algorithm 3 GPU Partitioning Sub-Algorithm

```

1: Inputs: List  $Y$  of selected hardware partition sizes, List  $x_k$  of the number of
   selection times of the selected hardware partition  $k$ , List  $CN$  of possible hardware
   partition configuration in GPU,  $N_{ic}$  number of partition  $i$  within hardware partition
   configuration  $c$ ,  $R_{ic}$  size of partition  $i$  within hardware partition configuration  $c$ ,
    $D_c$  number of partitions within hardware partition configuration  $c$ ,  $C_g$  all available
   resources of GPU
2: Output:  $PC$  list of optimal partition configurations from the selected hardware
   partitions
3: for ( $k = 1$  to  $Y$ ) do
4:   for ( $c = 1$  to  $CN$ ) do
5:      $\backslash \backslash x_k$  is the number of selection times of the selected partition  $k$ 
6:     while ( $x_k > 0$ ) do
7:       if partition  $k$  is in the hardware partition configuration  $c$  then
8:          $Minimize: \sum (C_g - \sum_{i=1}^{D_c} R_{ic} \cdot N_{ic} \cdot y_i) \cdot PC_c$ ;
9:         if option  $c$  fits to minimize the value then
10:           $y_i \leftarrow 1$ ;
11:           $PC_c \leftarrow PC_c + 1$ ;
12:           $x_k \leftarrow x_k - N_{ic}$ ;
13:        else
14:           $y_i \leftarrow 0$ ;
15:        end if
16:      end if
17:    end while
18:  end for
19: end for

```

the algorithm loops through all selected partitions (line 3). It examines every possible CN option (line 4), attempting to allocate the required number of partitions to the most suitable CN configuration (lines 5-14). We formulate Equation (16) to select the most appropriate CN for each chosen hardware partition, with the goal of minimizing resource waste in each CN . In this formula, the variable PC_c represents how often a specific hardware partition configuration, denoted as $c \in CN$, is chosen. Additionally, C_g refers to the total available GPU resources.

$$Minimize : \sum_{c=1}^{CN} \left(C_g - \sum_{i=1}^{D_c} R_{ic} \cdot N_{ic} \cdot y_i \right) \cdot PC_c \quad (16)$$

The algorithm assesses each configuration option $c \in CN$ through this formula to evaluate its effectiveness in minimizing Equation (16) (as indicated in line 8). For each configuration option c , the variable y_i is set to 1 if choosing partition i within option c contributes to minimizing Equation (16), and 0 if it does not (lines 9-16). By the end of this process, the GPU Partition Manager collects a list of optimal partition configurations.

The GPU partitioning sub-algorithm can be viewed as a solution for a variant of the NP-hard problem. To solve it optimally, the algorithm employs a solver, which introduces binary variables y_i for partition selection and integer variables PC_c for Equation (16). In the worst case, the solver must consider all possible configuration options $c \in CN$, resulting in an exponential-time search. The worst-case time complexity is thus:

$$O(2^{CN}), \quad (17)$$

where $|CN|$ is the number of admissible GPU partition configuration options. However, modern solvers significantly reduce the practical search space. As a result, the empirical runtime often scales much closer to polynomial in the number of

Algorithm 4 Task Placement Sub-Algorithm

```

1: Inputs:  $PC$  list of optimal partition configurations from the selected hardware
   partitions,  $K$  hardware partition queues for the grouped tasks, List  $GP$  of list of
   grouped tasks,  $G$  all available GPUs
2: for ( $g = 1$  to  $GP$ ) do
3:    $MTET_g \leftarrow \text{Maximize} : (T_{mn} \text{ of all the tasks in } g)$ ;
4: end for
5: for ( $i = 1$  to  $K$ ) do
6:   Sort the list of  $GP_i$  in the queue  $i$  based on the  $MTET$ ;
7: end for
8: for ( $p = 1$  to  $PC$ ) do
9:    $PL_p \leftarrow$  calculate based on Equation.(19)
10: end for
11: for ( $p = 1$  to  $PC$ ) do
12:   Sort the list of  $PC$  based on the  $PL_p$ ;
13: end for
14: while ( $GP > 0$ ) do
15:   for ( $n = 1$  to  $G$ ) do
16:     if  $n$  GPU  $n$  has available resources then
17:       choose the  $PC$  with high-priority;
18:       for all partitions in selected  $PC$  do
19:         choose the  $GP$  with high-priority;
20:          $GP \leftarrow GP - 1$ ;
21:       end for
22:     end if
23:   end for
24: end while

```

configurations and selected partitions. In practice, the solver-assisted runtime of Algorithm 3 can be approximated as

$$O(Y \cdot CN), \quad (18)$$

where Y is the number of selected partitions received from the Task Grouping sub-algorithm. Therefore, while the GPU Partitioning sub-algorithm exhibits exponential worst-case complexity, its solver-assisted runtime complexity is closer to linear in practice, which ensures feasibility for real-world deployments.

C. Task Placement

In the final stage, the time-based fairness allocation strategy is employed. The Task Placement sub-algorithm receives two key inputs: a list of grouped tasks assigned to k hardware partition queues by the Task Grouping sub-algorithm and a set of optimal partition configurations determined by the GPU Partitioning sub-algorithm. Its primary function is to allocate the earliest available GPU time slot for executing the grouped tasks. Additionally, the algorithm prioritizes hardware partition combinations for partitioning available GPUs sequentially, ensuring efficient allocation, especially when the number of selected partition combinations exceeds the available GPUs. We have designed a heuristic approach to manage the Task Placement sub-algorithm efficiently, as described in Algorithm 4. To promote fairness, the algorithm first determines the Maximum Task Execution Time (MTET) for each grouped task (lines 2–4), representing the longest duration a group occupies a partition. Next, it sorts the grouped tasks within each hardware partition queue based on their MTET values, giving priority to those with shorter MTETs for earlier scheduling (lines 5–7). This method helps to minimize task completion times by minimizing overall waiting periods for all tasks.

Once the priority of grouped tasks has been established, the next step involves partitioning the GPUs based on the selected partition combinations (PC) designated to host these tasks.

To effectively prioritize the PCs, we calculate and assign a priority level (PL) to each one and organize them accordingly (as outlined in lines 8–10). Each partition combination is made up of specific hardware partitions, each linked to a queue of grouped tasks. This phase involves a careful selection of the necessary grouped tasks for each partition within the PC, drawing them from the corresponding queue. The tasks in each partition's queue are sorted so that those with higher priority are selected first, promoting a fair scheduling process. Once enough grouped tasks are allocated for each PC, we compute their priority level (PL) using the following formula:

$$PL_p = (CCV_p + SPCO_p) \quad \forall p \in PC \quad (19)$$

The formula incorporates SPCO, short for Switch Partition Combination Overhead, which accounts for the time and computational effort needed to transition between different GPU partition configurations. Changing a specific Partition Combination (PC) involves three steps: (i) removing the existing partitions, (ii) restarting the GPU driver, and (iii) creating the new target partitions. We define the SPCO for target partition combination p as:

$$SPCO_p = (T_{remove} + T_{restart} + T_{create}(p)) \quad p \in PC \quad (20)$$

where T_{remove} is the time to remove all partitions, $T_{restart}$ is the driver restart time, and $T_{create}(p)$ is the time to instantiate the target partition combination p . These values can be measured using standard GPU management interfaces, for example, in NVIDIA devices, can be obtained through NVML/DCGM or nvidia-smi. Since modifying the current partition setup may necessitate a driver restart, it can significantly impact the efficiency of the method. To mitigate this, SPCO is introduced as a factor that lowers the priority of a specific PC, thereby minimizing frequent reconfigurations and enhancing overall performance. Accordingly, SPCO captures the reconfiguration overhead of switching GPU partition combinations, while the waiting-time metric $w_{mn}(t)$ already accounts for the delay when a task is paused due to eviction until it resumes. Together, these mechanisms ensure that eviction and rescheduling overheads are incorporated into the algorithm's performance evaluation. Additionally, the Consensus Characteristic Values (CCV) of a partition combination assess how effectively a grouped set of tasks within a PC contributes to fair resource allocation. CCV is determined by identifying the maximum Mean Task Execution Time (MTET) across partitions within a given PC. By prioritizing PCs with lower MTET, CCV helps reduce the overall waiting time for all tasks, leading to more efficient scheduling.

The task allocation algorithm assigns a PL to each PC and arranges them in descending order based on their PL values (lines 11–13). Next, the available GPUs are partitioned sequentially based on the prioritized PCs (lines 15–17). The method then organizes the scheduled tasks within the GPU partitions according to their assigned priority (lines 18–20). As GPU resources become available, the algorithm iteratively cycles through the PCs, determining the next configuration for GPU partitioning until all task groups have been fully processed. Once a grouped task is assigned through this procedure, it executes within the designated hardware partition

of the selected GPU until completion. Tasks remain pinned to that GPU unless Routine 2 triggers a repartitioning step, in which case they may be evicted and rescheduled.

The computational complexity of the Task Placement sub-algorithm is dominated by the sorting phases. If GP is the number of grouped tasks, K the number of partition queues, and PC the number of partition combinations, the time complexity can be expressed as:

$$O(GP \cdot \log GP + PC \cdot \log PC + GP \cdot PC), \quad (21)$$

where the first two terms correspond to sorting grouped tasks and partition combinations, and the last term accounts for the iterative allocation of tasks across partition combinations. Therefore, the Task Placement sub-algorithm exhibits polynomial time complexity, scaling well with the number of grouped tasks and partition combinations. This ensures that it can be efficiently applied in real-world scheduling scenarios.

D. Dynamic Workload Adaptation

Beyond the three core sub-algorithms, it is important to explain how the proposed method adapts to dynamic workload conditions, such as sudden spikes in task arrivals or abrupt changes in resource demands. Our design incorporates several mechanisms to ensure responsiveness and stability in these scenarios. First, we utilize a two-routine evaluation mechanism that provides both immediate task allocation and global optimization. When idle resources are available, Routine 1 quickly assigns new tasks with minimal overhead. In contrast, Routine 2 reevaluates all resources and tasks to maintain fairness when spikes exceed our idle capacity. Second, the monitoring module continuously updates the system state by tracking task progress and GPU partition status. This continuous monitoring enables the gradual invocation of the solver, reducing decision latency and facilitating rapid adaptation without the need to recompute everything from scratch. Finally, the fairness-driven task placement sub-algorithm prioritizes short tasks while minimizing disruptive repartitioning overhead by utilizing metrics such as MTET and SPCO. This approach ensures stable and fair scheduling, even when workload conditions change abruptly. Together, these mechanisms guarantee that our proposed method remains responsive, efficient, and fair under volatile workload conditions.

In addition, we note that dynamic reallocation and repartitioning may introduce latency overhead. This overhead comes from two sources: (i) solver execution time during re-optimization and (ii) GPU reconfiguration when switching partition combinations. To mitigate the first source, our framework performs incremental re-optimization using updated system states rather than recomputing from scratch. Modern solvers, such as Google OR-Tools, significantly reduce the effective search space, enabling decisions to be made within milliseconds for practical workloads. For the second source, we explicitly account for GPU reconfiguration delays through the SPCO metric in the Task Placement sub-algorithm, which prevents excessive repartitioning. Together, these mechanisms keep the latency of reallocation and repartitioning low relative to overall task execution times, ensuring that the framework remains responsive even under dynamic workload conditions.

V. PERFORMANCE EVALUATION

In this section, we evaluate the proposed methods by conducting extensive experiments across various scenarios. To assess the effectiveness and scalability of our approach, we developed a simulator implementing the three sub-algorithms tailored for a multi-tenant cloud setting. The simulator includes several components: the task grouping module, GPU partitioning module, task placement module, monitoring module, and solver service. The solver serves as a solver for both Task grouping and GPU partitioning. The solver service in the simulator utilizes the CBC MIP solver from the Google OrTools Solver library, which assists in determining the optimal solution for our formulated model. To evaluate real-world scenarios, we incorporated the Alibaba cluster-trace-gpu-v2023 dataset [48] in our evaluation, which captures AI and machine learning tasks with diverse resource specifications in a heterogeneous GPU cluster. The outcomes of the simulator are compared with four methods: MISO [7], SJF, and FGD [33]. Subsequently, we will outline the simulation setup, detailing the experimental setting and implementation specifics. Afterwards, we will discuss the evaluation outcomes and compare the performance of our proposal with existing solutions.

A. Experimental Setup

The simulator models a cloud data center environment with a cluster consisting of up to 64 NVIDIA A100-SXM4-40GB GPUs [44]. It is important to highlight that the proposed method relies on the NVIDIA GPU for testing. Our evaluation is conducted using an NVIDIA A100 GPU equipped with MIG support, which allows hardware partitioning. However, the proposed method is not restricted to GPUs with hardware partitioning capabilities. On GPUs that lack this feature, our method can function in two ways: either by dedicating the entire GPU to a single user, which ensures strict isolation but may lead to underutilization, or by utilizing software-based mechanisms such as MPS, which enhances utilization but offers weaker isolation. In both scenarios, our method maintains fairness. We provide a detailed discussion of the tradeoffs in Section V-C. The real-world Alibaba cluster-trace-gpu-v2023 dataset contains a total of 7064 GPU requests logged over 145 days. We have analyzed these requests to illustrate active requests throughout the duration. Accordingly, Figure 3 visualizes the distribution of the amount of GPU resource requests over time, illustrating the fluctuations in request volume during the observed period. The x-axis represents time in days, converted from seconds, while the y-axis indicates the number of GPU requests at each time. The continuous line graph highlights trends in GPU requests, with peaks representing periods of high demand and troughs indicating lower activity. This visualization provides insights into the temporal behaviour of GPU requests.

To ensure a comprehensive evaluation of scheduling efficiency, we analyzed two key real-world scenarios that assess the impact of user demand and GPU capacity dynamics:

- **Scaling Users:** We varied the number of active users from 1 to 8, simulating different levels of concurrent

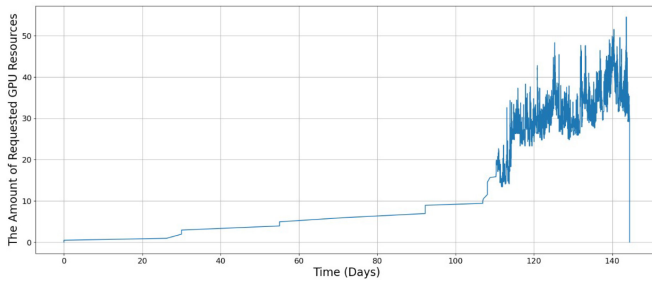


Fig. 3. Distribution of GPU Resource Requests Over Time in Alibaba.

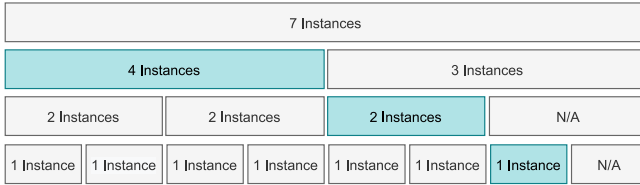


Fig. 4. A100 MIG Instance configuration policy.

demand on the system. This scenario helps assess how effectively the scheduling mechanism handles increasing user requests and resource contention.

- **Scaling GPUs:** We adjusted the available GPU resources from 2 to 64, allowing us to analyze the system's adaptability to changes in GPU availability. This scenario evaluates how effectively the scheduling approach utilizes varying GPU capacities to meet user demands.

A midpoint threshold of 0.5 ($\text{Thr}=0.5$) is established to balance resource utilization and fair task scheduling.

Using an A100 GPU with MIG technology enables the GPU to be divided into a maximum of 7 hardware partitions (instances), which can be sized at 1/7, 2/7, 3/7, 4/7, or the entire 7/7 of the GPU. Each partition possesses its own distinct GPU memory and Streaming Multiprocessor (SM). With the option for seven instances, the A100 can be arranged in a versatile manner, allowing for a single GPU with seven instances totalling 98 SMs and 40 GB of memory or various smaller instances. These configurations can consist of one instance with 14 SMs and 5 GB, two instances with 28 SMs and 10 GB, three instances with 42 SMs and 20 GB, or four instances with 56 SMs and 20 GB, providing adaptable computational capabilities.

In addition to the limitations on the size of instances, there are restrictions on the configuration of MIG instances; only a specific set of instance combinations is permitted in A100. Figure 4 showcases the allowed instances along with the configuration policy for A100 MIG instances. The instance selection policy for a valid combination is such that the first instance is chosen from the left side. Once a partition is selected, subsequent selections must avoid instances located below it. As a result, there are 19 different combinations of supported instances [45]. Given that a GPU unit represents the smallest partition in an A100 GPU, and each GPU has a capacity of 7 units, the overall GPU capacity for this experiment is up to 448 units distributed across 64 GPUs. The simulator, encompassing resource allocation and scheduling

services, is developed using C# 11 and dotnet 8.0.204 on a desktop with an Intel Core i7-10700 CPU clocked at 2.90GHz and 16 GB of memory.

B. Results

This section evaluates the effectiveness of the proposed method in allocating resources while minimizing both task completion time and resource underutilization. We will assess the proposed method in two real-world scenarios: scaling users and GPUs. To demonstrate its advantages in terms of efficiency and scalability, we will compare the proposed method with FGD [33], SJF, and MISO [5]. It is important to note that our simulation results are not directly comparable to earlier research that did not consider resource isolation, which is a critical factor in resource management and user security within a cloud environment.

Before presenting the experimental results, Table II offers a qualitative comparison of our proposed method against the baseline methods: FGD [33], SJF, and MISO [5]. This comparison encompasses several dimensions, including triggering mechanism, execution location, vendor dependence, hardware/software category, overhead, and qualitative strengths and weaknesses. Unlike the baseline methods, our framework integrates both hardware and software techniques, incorporates fairness metrics, and explicitly considers reconfiguration overhead (SPCO). This makes our approach both adaptive and vendor-agnostic, while maintaining the flexibility to dynamically repartition and co-locate tasks. Next, we will present the quantitative evaluation results for two scenarios: user scaling and GPU scaling.

1) *User Scaling Scenario:* In this scenario, the number of active users is increased exponentially from 1 to 8 to simulate rising demand as the number of tasks scales. We evaluated how different scheduling mechanisms handle resource contention by using 7064 requests based on a real-world Alibaba dataset, as shown in Figure 3. To focus on stable task demands, we first removed the initial low-demand period. Additionally, to ensure that the requests come from different users and to simulate concurrent activity, we divided the request duration by the number of users and adjusted each user's timeline to start from zero. This approach guarantees that, during the experimental period, all users are actively submitting requests in parallel, reflecting a realistic multi-tenant GPU scheduling scenario. Figure 5 illustrates an example with five concurrent users, where each color represents the task requests from a specific user.

In this scenario, each user can utilize up to 56 GPU units of resources from a total pool of 8 GPUs. The execution times for each task may vary, and the resource requirements for each task range from 1 to 7 units. Figure 6 compares resource underutilization across FGD, SJF, and MISO as the number of active users increases. Resource underutilization occurs when resources are not used efficiently or when fragmentation occurs, leading to higher costs and lower productivity. We calculate this resource underutilization by measuring the amount of idle resources, including fragmented parts, during the execution of tasks. This value can be obtained by multiplying

TABLE II
QUALITATIVE COMPARISON OF SCHEDULING APPROACHES

Algorithm	Triggering / Decision Timing	Execution Location	Vendor / HW Dependence	HW / SW Category	Overhead Characteristics	Key Strengths / Weaknesses
Proposed method	Event-driven (task arrivals & completions)	Cluster-level scheduler	None (agnostic)	Hybrid (HW+SW)	Solver runtime (task grouping, partitioning) + GPU repartitioning	High utilization, fairness-aware; supports isolation and dynamic repartitioning; Reduces fragmentation
MISO [7]	Profiling with MPS and partition decision	Cluster-level scheduler	Yes (requires NVIDIA MIG/MPS support)	HW-only	MPS profiling + partition switching cost; limited partition flexibility causes fragmentation	Effective on MIG GPUs; limited generality beyond NVIDIA MIG; less flexible co-location.
SJF	Event-driven (task arrivals)	Cluster-level scheduler	None (agnostic)	SW-only	sorting by estimated job length	Very low overhead; ignores resource demand; high fragmentation; long jobs may starve.
FGD [33]	Event-driven	Cluster-level scheduler	None (agnostic)	SW-only	fragmentation monitoring + fairness adjustments	Reduces fragmentation; no isolation; no dynamic repartitioning; cannot pack across users under isolation constraints.

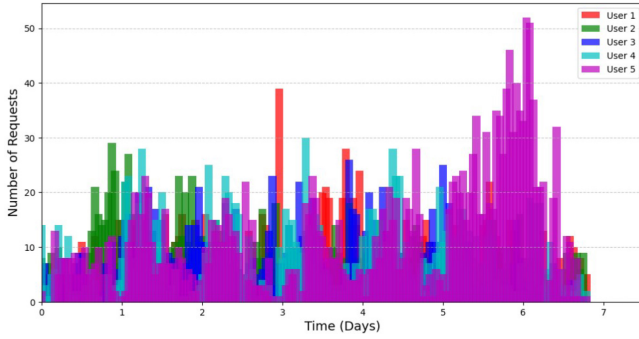


Fig. 5. Distribution of Requests Over Time by User.

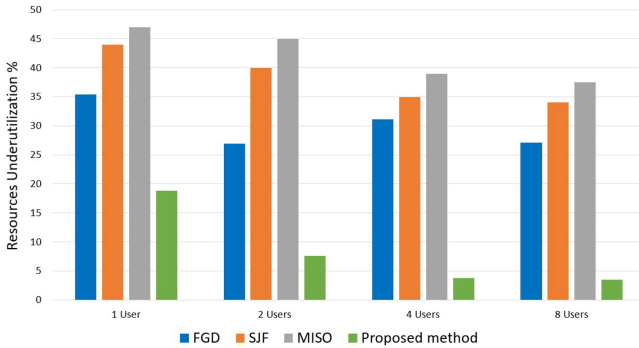


Fig. 6. Resources Underutilization During Task Execution for different numbers of users in the cluster.

the number of idle units by their duration of idleness.

$$IdleCost = \sum_i (IdleUnits_i \times IdleTime_i) \quad (22)$$

By summing these values, we can gain insights into the total amount of unused resources during the execution of all tasks. To express this as a percentage, we normalize by the total available GPU capacity over the entire experiment, which is the total number of GPU units (TGU) multiplied by the total execution time (TET):

$$Underutilization\% = \frac{IdleCost}{TGU \times TET} \times 100\% \quad (23)$$

As shown in Figure 6, the results indicate that the SJF scheduling algorithm exhibits the highest levels of resource waste. This is primarily due to its focus on prioritizing

shorter tasks without considering their resource requirements. In addition, while MISO employs hardware-based partitioning, it suffers from inefficiencies arising from limited flexibility in partitioning, which leads to GPU fragmentation. MISO also emphasizes finding the optimal partition for each task, but does not consider the constraints of combining different partitions, resulting in further GPU fragmentation. In contrast, our proposed method utilizes software-based partitioning to co-locate a user's trusted task within a single hardware partition, maximizing resource utilization. Furthermore, incorporating a solver service in the GPU partitioning sub-algorithm significantly reduces idle resources. Comparison results demonstrate that, although FGD outperforms both SJF and MISO, our proposed method's dynamic adjustment and rescheduling tasks lead to significant improvements in GPU resource usage. As illustrated in the Figure, the comparison results show an enhancement in resource utilization compared to the SJF, FGD, and MISO methods, with a guaranteed minimum improvement of 88.10%.

The maximum normalized task completion time is shown in Figure 7, illustrating the effectiveness of the proposed method in reducing task execution time as user demand increases. Our approach results in a lower maximum normalized task completion time compared to both SJF and MISO. While SJF prioritizes shorter tasks to minimize waiting time, it can create resource bottlenecks in a shared GPU environment, ultimately increasing overall task completion time. This happens because executing shorter tasks with high resource demands can leave fewer resources available, blocking the execution of other tasks. In contrast, our proposed fair scheduling method avoids such blocking by considering both the resource requirements and the expected execution time of tasks in the Task Grouping sub-algorithm. In this sub-algorithm, the strategy of grouping tasks with similar execution times into a single partition is crucial for reducing overall task completion time. The Task Placement sub-algorithm, which sorts the grouped tasks along with the list of partition configurations, provides significant advantages in terms of task completion times. On the other hand, constrained by limited partitioning options, MISO fails to optimize task placement, resulting in prolonged execution times. While the FDG is designed for fair resource allocation and performs better than SJF and MISO, it still lacks the efficiency of our proposed approach. The reason is that the

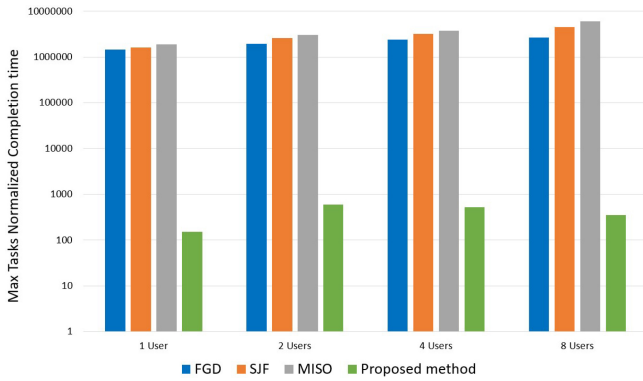


Fig. 7. Max Tasks Normalization Time During Task Execution for different numbers of users in the cluster.

proposed approach utilizes dynamic rescheduling, task grouping, and optimized partition placement to improve scheduling efficiency and minimize delays. Moreover, the proposed routines in our approach enable the repartitioning and eviction of currently running tasks, leading to fair allocation and shorter task completion times.

2) *GPU Scaling Scenario*: To evaluate how well scheduling strategies adapt to GPU resource availability, we increased the number of available GPUs exponentially from 2 to 64. This approach enables us to assess how effectively each approach utilizes the additional GPU capacity to meet user demands. Figure 8 illustrates the underutilization of resources as GPU capacity increases. The results of the SJF scheduling method indicate inefficient GPU usage because it does not schedule tasks according to available resources, leading to significant idle GPU capacity. Similarly, the MISO method experiences GPU fragmentation due to its dependence on predefined hardware partitions, which restricts its ability to allocate resources efficiently. While the FDG approach performs better than both SJF and MISO, it still does not achieve full utilization. This is primarily due to its fair allocation strategy, which does not always prioritize optimal resource usage. In contrast, our proposed method dynamically adjusts task groupings based on the available GPU resources, significantly reducing underutilization. Our proposed approach still encounters some underutilization because the grouped tasks may not fully utilize a hardware partition. Consequently, some resources may remain idle, potentially leading to delays in task execution.

Finally, Figure 9 presents the max normalized task completion time as GPU resources scale. While increasing GPU capacity generally improves task execution times, the effectiveness of this improvement depends on the scheduling approach. SJF and MISO fail to fully leverage additional GPU resources, leading to higher completion times. FDG performs better than these methods but remains suboptimal compared to the proposed approach, which employs a resource-aware task grouping strategy to efficiently distribute tasks across available GPUs. As a result, the proposed method achieves faster task execution. It is worth mentioning that as the number of GPUs increases, such as in the case of 64 GPUs, the performance of the proposed approach stabilizes and shows similar results because there are sufficient resources to handle the load. In

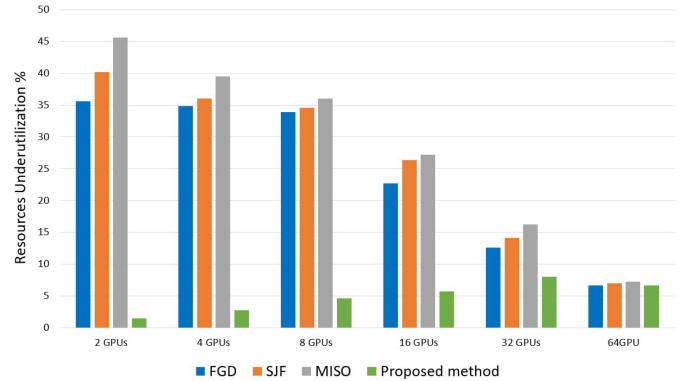


Fig. 8. Resources Underutilization During Task Execution for different numbers of GPUs in the cluster.

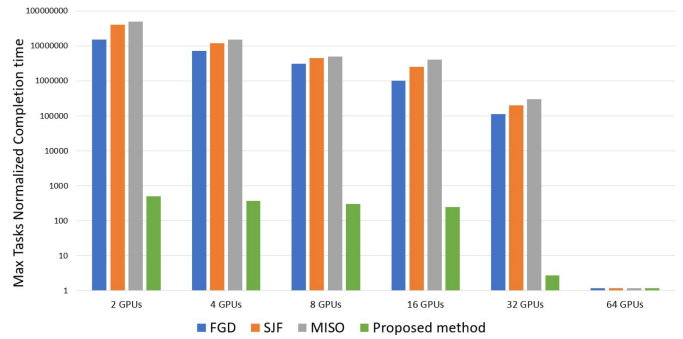


Fig. 9. Max Tasks Normalization Time During Task Execution for different numbers of GPUs in the cluster.

this scenario, the scheduling mechanism has less impact, as the system has ample resources to meet the demand, and task completion times are already optimized by the availability of resources.

C. Discussions

The evaluation results show that using the combination of hardware and software partitioning in the proposed resource allocation and task scheduling method can significantly improve resource utilization, fairness and isolation. While our method is compatible with GPUs that lack hardware partitioning, this compatibility comes with several important considerations. In such environments, there are generally two ways to achieve isolation: (i) treating the entire GPU as a single hardware partition and assigning it exclusively to one user, which provides strict isolation but leads to significant underutilization, or (ii) relying on software-only partitioning mechanisms such as MPS, which allow higher utilization but offer weaker isolation. In both cases, our proposed method continues to enforce fairness in scheduling across different workloads. However, software-only approaches cannot fully prevent performance interference and dedicating the whole GPU sacrifices efficiency. Therefore, while our method remains broadly applicable to various GPUs, the guarantees of fairness, isolation, and utilization depend on the chosen strategy. Therefore, while our method is broadly applicable to various GPUs, the guarantees of fairness, isolation, and utilization will depend on the chosen strategy. We believe that

developing hybrid mechanisms that balance utilization and isolation without hardware support is an important area for future research.

Another important consideration is task heterogeneity. In our current model, a task is defined as the smallest schedulable unit mapped to a GPU partition, which keeps the framework task-agnostic. However, workloads such as real-time inference, batch inference, or training can exhibit very different latency and throughput requirements. While our current algorithms schedule all tasks uniformly based on their resource needs and estimated execution times, task types like these can be incorporated by adjusting the scheduling parameters. For example, assigning more rigid waiting-time bounds or higher scheduling priority to latency-sensitive inference tasks, while allowing batch training tasks to tolerate longer delays. Moreover, as part of our ongoing work, we are extending the framework to explicitly handle task priorities, preempt long-running tasks when necessary, and further improve fairness across heterogeneous workloads.

VI. CONCLUSION

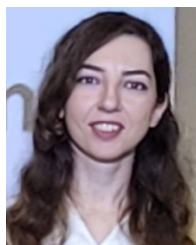
In this paper, we propose an efficient GPU resource allocation technique integrated with an optimized task scheduling solution to address the challenges of underutilized GPU resources and suboptimal task completion times in multi-tenant cloud environments. Our approach leverages a combination of software-based partitioning and LP solver optimization to maximize GPU resource utilization while minimizing task completion time. The experimental results demonstrate that our proposed method guarantees at least an 88% improvement in resource utilization, achieving significant reductions in resource wastage compared to the FGD, SJF and MISO techniques. Furthermore, our approach shows significant improvement in maximum normalized task completion time. We demonstrated that the proposed method outperforms existing techniques through two scaling scenarios: scaling users and scaling GPUs. In the user scaling scenario, it adapts effectively to increasing demand, while in the GPU scaling scenario, it maximizes resource utilization as the number of GPUs grows. Overall, our proposed method offers substantial improvements over existing techniques, ensuring both optimal resource efficiency and minimized task delays in cloud-based GPU scheduling environments.

REFERENCES

- [1] Q. Weng et al., "MLaaS in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters," in *Proc. 19th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2022, pp. 945–960.
- [2] H. Zhao et al., "HiveD: Sharing a GPU cluster for deep learning with guarantees," in *Proc. 14th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2020, pp. 515–532.
- [3] K. Seshadri, K. Sindhu, S. N. Bhattu, and C. Kollengode, "Design and evaluation of a hierarchical characterization and adaptive prediction model for cloud workloads," *IEEE Trans. Cloud Comput.*, vol. 12, no. 2, pp. 712–724, Apr.–Jun. 2024.
- [4] S. S. Shubha, H. Shen, and A. Iyer, "{USHER}: Holistic interference avoidance for resource optimized {ML} inference," in *Proc. 18th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2024, pp. 947–964.
- [5] C. Zhao, W. Gao, F. Nie, and H. Zhou, "A survey of GPU multitasking methods supported by hardware architecture," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 6, pp. 1451–1463, Jun. 2022.
- [6] H. Sedighi, D. Gehberger, and R. Glitho, "Workload-aware dynamic GPU resource management in component-based applications," in *Proc. IEEE Int. Conf. Cloud Eng. (IC2E)*, 2022, pp. 213–220.
- [7] B. Li, T. Patel, S. Samsi, V. Gadepally, and D. Tiwari, "MISO: Exploiting multi-instance GPU capability on multi-tenant GPU clusters," in *Proc. 13th Symp. Cloud Comput.*, 2022, pp. 173–189.
- [8] C. Tan et al., "Serving DNN models with multi-instance GPUs: A case of the reconfigurable machine scheduling problem," 2021, *arXiv:2109.11067*.
- [9] Z. Ye et al., "ASTRAEA: A fair deep learning scheduler for multi-tenant GPU clusters," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 11, pp. 2781–2793, Nov. 2022.
- [10] J. Y.-T. Leung, M. Dror, and G. H. Young, "A note on an open-end bin packing problem," *J. Schedul.*, vol. 4, no. 4, pp. 201–207, 2001.
- [11] H. Sedighi, F. Wuhib, and R. Glitho, "Efficient task scheduling and allocation of GPU resources in clouds," in *Proc. 15th Int. Conf. Netw. Future (NoF)*, 2024, pp. 115–123.
- [12] C. Li, A. Zigerelli, J. Yang, Y. Zhang, S. Ma, and Y. Guo, "A dynamic and proactive GPU preemption mechanism using checkpointing," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 1, pp. 75–87, Jan. 2020.
- [13] S. Garg, K. Kothapalli, and S. Purini, "Share-a-GPU: Providing simple and effective time-sharing on GPUs," in *Proc. IEEE 25th Int. Conf. High Perform. Comput. (HiPC)*, 2018, pp. 294–303.
- [14] C. Hartmann and U. Margull, "GPUart—an application-based limited preemptive GPU real-time scheduler for embedded systems," *J. Syst. Archit.*, vol. 97, pp. 304–319, Aug. 2019.
- [15] I. Tanasic, I. Gelado, J. Cabezas, A. Ramirez, N. Navarro, and M. Valero, "Enabling preemptive multiprogramming on GPUs," *ACM SIGARCH Comput. Archit. News*, vol. 42, no. 3, pp. 193–204, 2014.
- [16] B. Wu, X. Liu, X. Zhou, and C. Jiang, "Flep: Enabling flexible and efficient preemption on GPUs," *ACM SIGPLAN Notices*, vol. 52, no. 4, pp. 483–496, 2017.
- [17] Y. Liang, H. P. Huynh, K. Rupnow, R. S. M. Goh, and D. Chen, "Efficient GPU spatial-temporal multitasking," *IEEE Trans. Parallel Distrib. Syst.*, vol. 26, no. 3, pp. 748–760, Mar. 2015.
- [18] P. Aguilera, K. Morrow, and N. S. Kim, "QoS-aware dynamic resource allocation for spatial-multitasking GPUs," in *Proc. 19th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, 2014, pp. 726–731.
- [19] P. Aguilera, K. Morrow, and N. S. Kim, "Fair share: Allocation of GPU resources for both performance and fairness," in *Proc. IEEE 32nd Int. Conf. Comput. Design (ICCD)*, 2014, pp. 440–447.
- [20] "Multi-process service," NVIDIA. 2024. Accessed: Feb. 2024. [Online]. Available: https://docs.nvidia.com/deploy/pdf/CUDA_Multi_Process_Service_Overview.pdf
- [21] W. Gao, Z. Ouyang, P. Sun, and T. Zhang, "IceFrog: A layer-elastic scheduling system for deep learning training in GPU clusters," *IEEE Trans. Parallel Distrib. Syst.*, vol. 36, no. 6, pp. 1071–1086, Jun. 2025.
- [22] G. Lim, J. Ahn, W. Xiao, Y. Kwon, and M. Jeon, "Zico: Efficient GPU memory sharing for concurrent DNN training," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2021, pp. 161–175.
- [23] Y. Zhao et al., "MuxFlow: Efficient and safe GPU sharing in large-scale production deep learning clusters," 2023, *arXiv:2303.13803*.
- [24] M. Li et al., "EasyScale: Elastic training with consistent accuracy and improved utilization on GPUs," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2023, pp. 1–14.
- [25] J. Bakita and J. H. Anderson, "Hardware compute partitioning on NVIDIA GPUs for composable systems," in *Proc. 37th Euromicro Conf. Real-Time Syst. (ECRTS)*, 2025, pp. 1–21.
- [26] Z. Mo, H. Xu, and W. C. Lau, "Optimal resource efficiency with fairness in heterogeneous GPU clusters," in *Proc. 25th Int. Middleware Conf.*, 2024, pp. 36–48.
- [27] X. Li, L. Pan, S. Liu, and X. Meng, "A fair and efficient resource allocation algorithm for cloud rendering jobs," *IEEE Trans. Services Comput.*, vol. 18, no. 4, pp. 1869–1880, Jul./Aug. 2025.
- [28] B. Hu, X. Yang, and M. Zhao, "Energy-minimized scheduling of intermittent real-time tasks in a CPU-GPU cloud computing platform," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 8, pp. 2391–2402, Aug. 2023.
- [29] B. Wu, Z. Zhang, Z. Bai, X. Liu, and X. Jin, "Transparent GPU sharing in container clouds for deep learning workloads," in *Proc. 20th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2023, pp. 69–85.

- [30] A. Ghodsi, M. Zaharia, B. Hindman, A. Konwinski, S. Shenker, and I. Stoica, "Dominant resource fairness: Fair allocation of multiple resource types," in *Proc. 8th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2011, pp. 323–336.
- [31] J. Gu et al., "Tiresias: A GPU cluster manager for distributed deep learning," in *Proc. 16th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2019, pp. 485–500.
- [32] K. Mahajan et al., "Themis: Fair and efficient GPU cluster scheduling," in *Proc. 17th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2020, pp. 289–304.
- [33] Q. Weng et al., "Beware of fragmentation: Scheduling GPU-sharing workloads with fragmentation gradient descent," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2023, pp. 995–1008.
- [34] N. Feddal, G. Lipari, and H.-E. Zahaf, "Towards efficient parallel GPU scheduling: Interference awareness with schedule abstraction," in *Proc. 32nd Int. Conf. Real-Time Netw. Syst.*, 2025, pp. 82–93.
- [35] Z. Mo, H. Xu, and W. C. Lau, "Fast and fair training for deep learning in heterogeneous GPU clusters," in *Proc. 39th ACM Int. Conf. Supercomput.*, 2025, pp. 324–338.
- [36] U. P. Jakarbet, A. A. Khan, S. P. Suhas, R. M. Savithramma, and B. P. Ashwini, "Comparative analysis of various game theory algorithms for CPU and GPU resource allocation," in *Proc. 4th Int. Conf. Ubiquitous Comput. Intell. Inf. Syst. (ICUIIS)*, 2024, pp. 129–134.
- [37] A. Qiao et al., "Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning," in *Proc. 15th USENIX Symp. Oper. Syst. Design Implement.*, 2021, pp. 1–18.
- [38] X. Wang, Y. Li, F. Guo, Y. Xu, and J. C. S. Lui, "Dynamic GPU scheduling with multi-resource awareness and live migration support," *IEEE Trans. Cloud Comput.*, vol. 11, no. 3, pp. 3153–3167, Jul.–Sep. 2023.
- [39] J. Zhai, J. Bi, H. Yuan, J. Zhang, and R. Buyya, "Energy-efficient and latency-aware task offloading for industrial cloud-edge systems with heterogeneous CPUs and GPUs," *IEEE Internet Things J.*, vol. 12, no. 13, pp. 25757–25772, Jul. 2025.
- [40] S. Singh and R. Nasre, "Scalable and performant graph processing on GPUs using approximate computing," *IEEE Trans. Multi-Scale Comput. Syst.*, vol. 4, no. 3, pp. 190–203, Jul.–Sep. 2018.
- [41] Z. Fink, K. Parasyris, G. Georgakoudis, and H. Menon, "HPAC-offload: Accelerating HPC applications with portable approximate computing on the GPU," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2023, pp. 1–14.
- [42] V. Leon et al., "Approximate computing survey, part I: Terminology and software & hardware approximation techniques," *ACM Comput. Surveys*, vol. 57, no. 7, pp. 1–36, 2025.
- [43] "Resource management for pods and containers," 2023. [Online]. Available: <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>
- [44] Q. Chen, H. Yang, J. Mars, and L. Tang, "Baymax: QoS awareness and increased utilization for non-preemptive accelerators in warehouse scale computers," in *Proc. 21st Int. Conf. Archit. Support Programming Lang. Oper. Syst.*, 2016, pp. 681–696. [Online]. Available: <https://api.semanticscholar.org/CorpusID:13778068>
- [45] Q. Chen, H. Yang, M. Guo, R. S. Kannan, J. Mars, and L. Tang, "Prophet: Precise QoS prediction on non-preemptive accelerators to improve utilization in warehouse-scale computers," in *Proc. 22nd Int. Conf. Archit. Support Programming Lang. Oper. Syst.*, 2017, pp. 17–32. [Online]. Available: <https://api.semanticscholar.org/CorpusID:55216>
- [46] G. Yeung, D. Borowiec, R. Yang, A. Friday, R. Harper, and P. Garraghan, "Horus: Interference-aware and prediction-based scheduling in deep learning systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 1, pp. 88–100, Jan. 2022.

- [47] "Device-plugins," 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/device-plugins/>
- [48] "Alibaba cluster trace," 2023. [Online]. Available: <https://github.com/alibaba/clusterdata>



Hoda Sedighi received the B.Sc. degree in computer engineering from the Isfahan University of Technology, Iran, and the M.Sc. degree in computer engineering from Shahid Beheshti University, Iran. She is currently pursuing the Ph.D. degree with Concordia University, Montreal, QC, Canada, where she is a member of the Telecommunication Service Engineering Research Lab. Her research focuses on cloud computing and resource management, with emphasis on high-performance GPU computing, GPU sharing, and performance modeling for next-generation networks. She also works on resource orchestration and availability challenges in 5G and cloud-native telecom systems.



Fetahi Wuhib received the B.Sc. degree in electrical and computer engineering from Addis Ababa University, Ethiopia, and the M.Sc. degree in internetworking and the Ph.D. degree in telecommunications from the KTH Royal Institute of Technology, Sweden. He is a Master Researcher in the Compute and Software area with Ericsson Research, where he has worked since 2013. His work covers resource management for large-scale cloud platforms, including GPU sharing, as well as Cloud RAN and 5G Core dimensioning with a strong focus on availability.



Roch H. Glitho (Life Senior Member, IEEE) received the M.Sc. degree in business economics from the University of Grenoble, France, the M.Sc. degrees in pure mathematics and computer science from the University of Geneva, Switzerland, and the Ph.D. degree (Tech.Dr.) in teleinformatics from the Royal Institute of Technology (KTH), Stockholm, Sweden. He is a Full Professor and holds an Ericsson /ENCQOR-5G Senior Industrial Research Chair. He has held a Canada Research from 2010 to 2020. He has performed research in industry and has held several senior technical positions, such as a Senior Specialist, a Principal Engineer, and an Expert with Ericsson, Stockholm, Sweden, and Montréal, QC, Canada. He has served as the Editor-in-Chief for the *IEEE Communications Magazine* and *IEEE COMMUNICATIONS SURVEYS AND TUTORIALS*. He has also served as an IEEE Distinguished Lecturer.